

DATA 260 Homework 4 Report

GITHUB REPOSITORY: <https://github.com/rutujasaste06/data260-5825>

Student ID- 020065825

SID4- 5825

PORT_BASE - 8425

PREFIX - s5825

SEED - 5825

VERIFY_SEED – 265825

DOMAIN_ID - 1

DOMAIN - Clinical trial listings

HARDWARE - MacBook Pro, Apple M5 chip

LOCAL MODEL USED: qwen3:8b (Ollama)

TAGGED COMMIT HASH: b6e8261d384a7c9bf2f555aae7f38480417dc124

Outputs:

Created the database “s5825_rel”

```
mysql> CREATE DATABASE s5825_rel;
```

```
mysql> SHOW DATABASES;
```

```
+-----+
| Database      |
+-----+
| information_schema |
| mysql          |
| performance_schema |
| s5825_rel      |
| sys            |
+-----+
```

5 rows in set (0.019 sec)

Installed “cryptography” because MySQL 8 and newer uses a newer password login method that pymysql can't handle without it. As 'cryptography' package is required for sha256_password

Created 3 tables using models.py file for Trials,users and session token..

```

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "USE s5825_rel; SHOW TABLES; DESCRIBE trials;"
Enter password:
+-----+
| Tables_in_s5825_rel |
+-----+
| sessions            |
| trials              |
| users               |
+-----+

+-----+
| Field      | Type          | Null | Key | Default | Extra           |
+-----+
| id         | int           | NO   | PRI | NULL    | auto_increment |
| trial_title | varchar(255)  | NO   |     | NULL    |                 |
| nct_number | varchar(50)   | NO   |     | NULL    |                 |
+-----+
• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

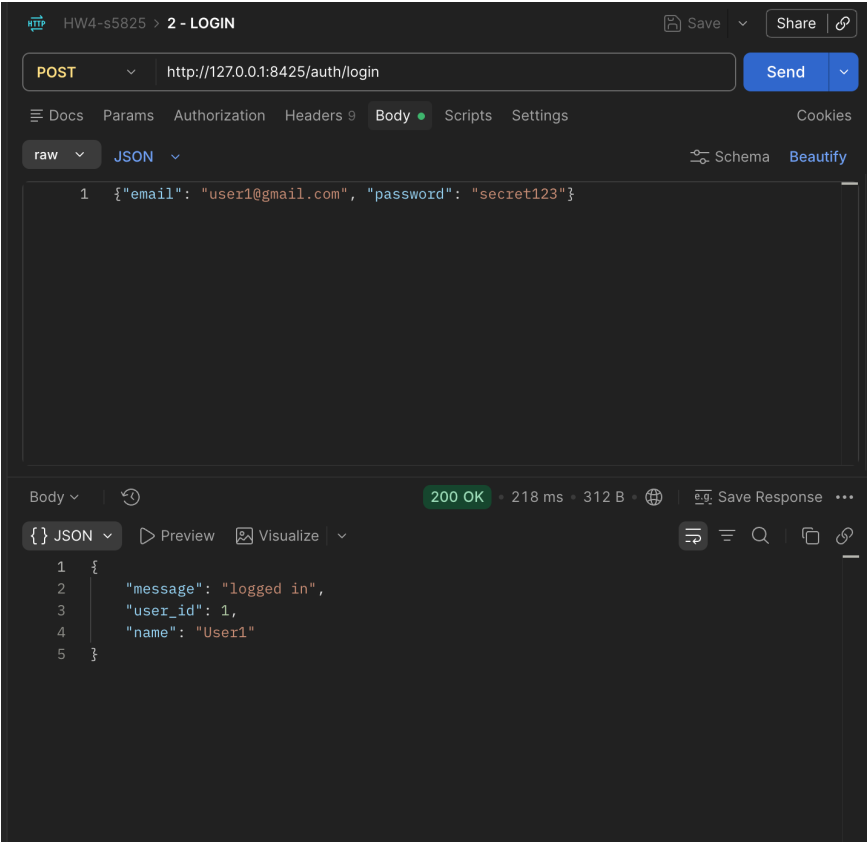
Part 2:

Register: (POST /auth/register) creates a new user account with a bcrypt-hashed password.

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: http://127.0.0.1:8425/auth/register
 - Body (raw): `{ "name": "User3", "email": "user3@gmail.com", "password": "secret123" }`
- Response:**
 - Status: 200 OK
 - Time: 197 ms
 - Size: 176 B
 - Body (JSON): `{ "id": 4, "name": "User3", "email": "user3@gmail.com" }`

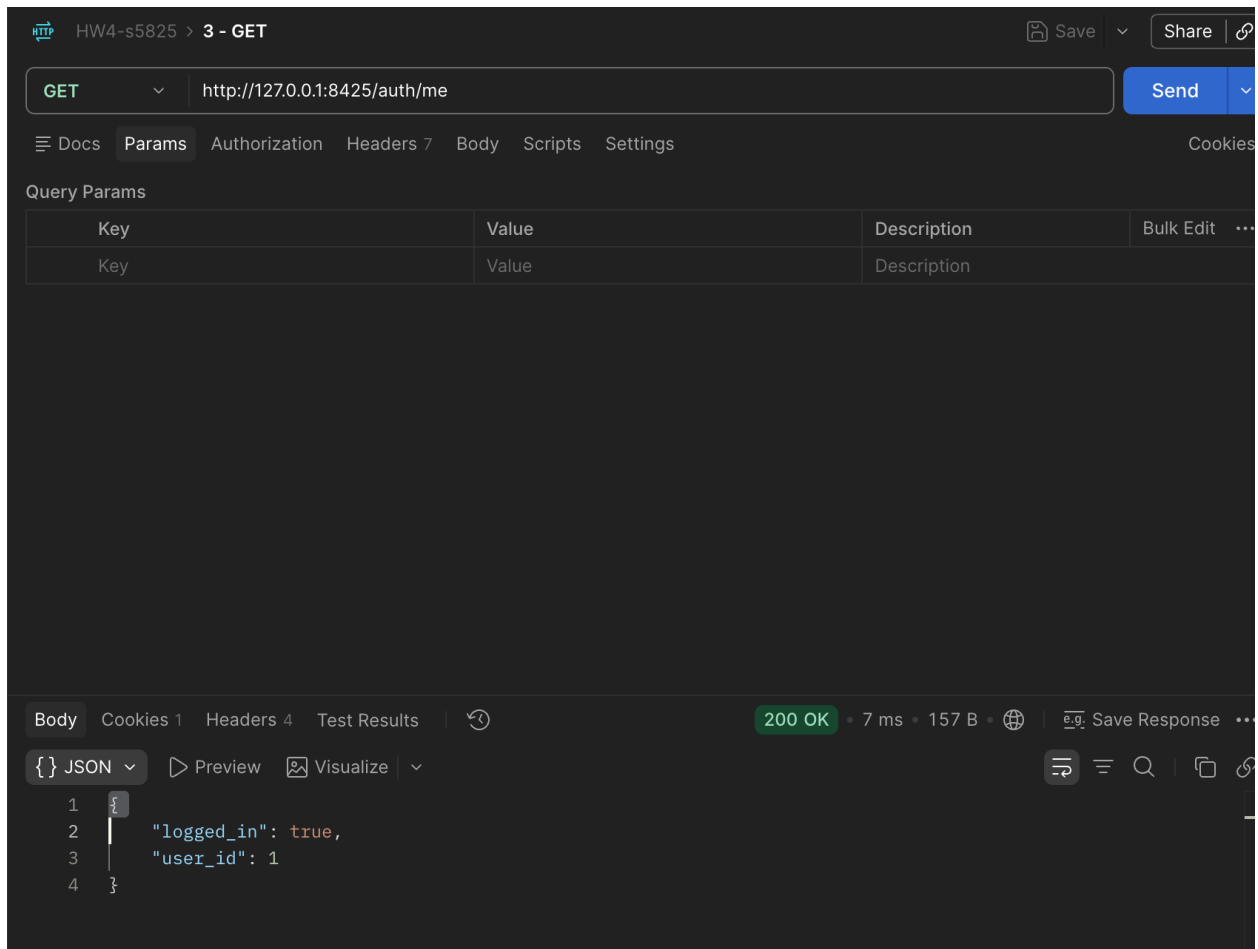
Called login



Session cookie:

Name	Value	Domain	Path	Expires	HttpOnly	Secure
session_id	ddb9f108ea50334...	127.0.0.1	/	Session	true	false

GET:



Trials CRUD operations:

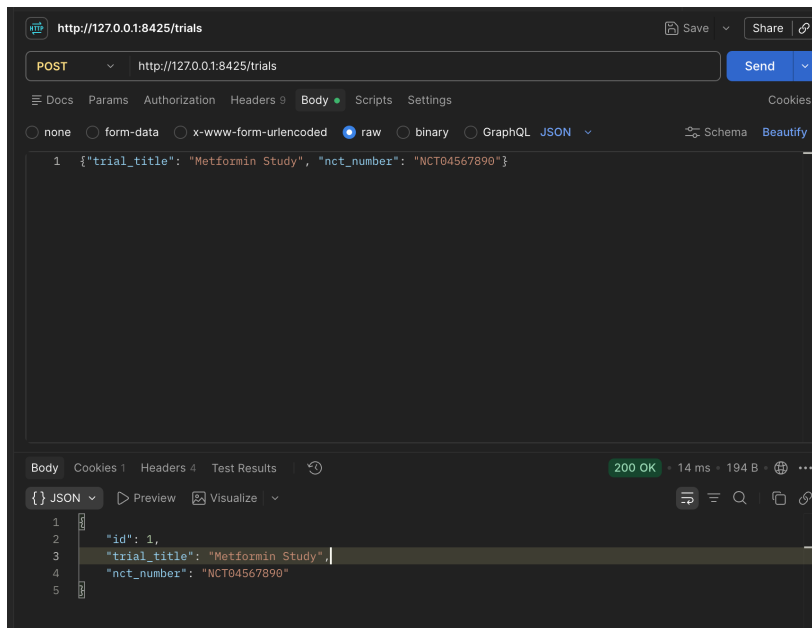
the session_id cookie is already stored in Postman from login

Request 1: Add a trial:

This request adds a new clinical trial record. The route in main.py first checks the session_id cookie through require_session, so only a logged-in user can add records. It then calls crud.create_trial, which inserts a row into the trials table in MySQL. The database assigns the auto-incremented id, and the API returns the saved record (id: 1, title "Metformin Study", NCT number "NCT04567890") with status 200 OK.

```
# ----- Trials CRUD (all need a valid session) -----
@app.post("/trials", response_model=schema.TrialOut)
def add_trial(payload: schema.TrialCreate, db: Session = Depends(get_db),
              _s=Depends(require_session)):
    return crud.create_trial(db, payload)
```

O/P:



Checked the trials inserted from the trials table in sql

```
(venv) rutujaste@Rutujaste-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.trials;"
Enter password:
+-----+-----+-----+
| id | trial_title | nct_number |
+-----+-----+-----+
| 1 | Metformin Study | NCT04567890 |
+-----+-----+-----+
```

Request 2: View all records (GET):

After logging in when the session stays active and when the trials list is called using GET method to view all records.

```
98 @app.get("/trials", response_model=list[schema.TrialOut])
99 def list_trials(db: Session = Depends(get_db), _s=Depends(require_session)):
100     return crud.get_trials(db)
101
```

HW4-s5825 > GET 5 - View all Trials

GET

http://127.0.0.1:8425/trials

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies 1

Headers 4

Test Results

200 OK

4 ms

280 B

Save Response

{ } JSON

Preview

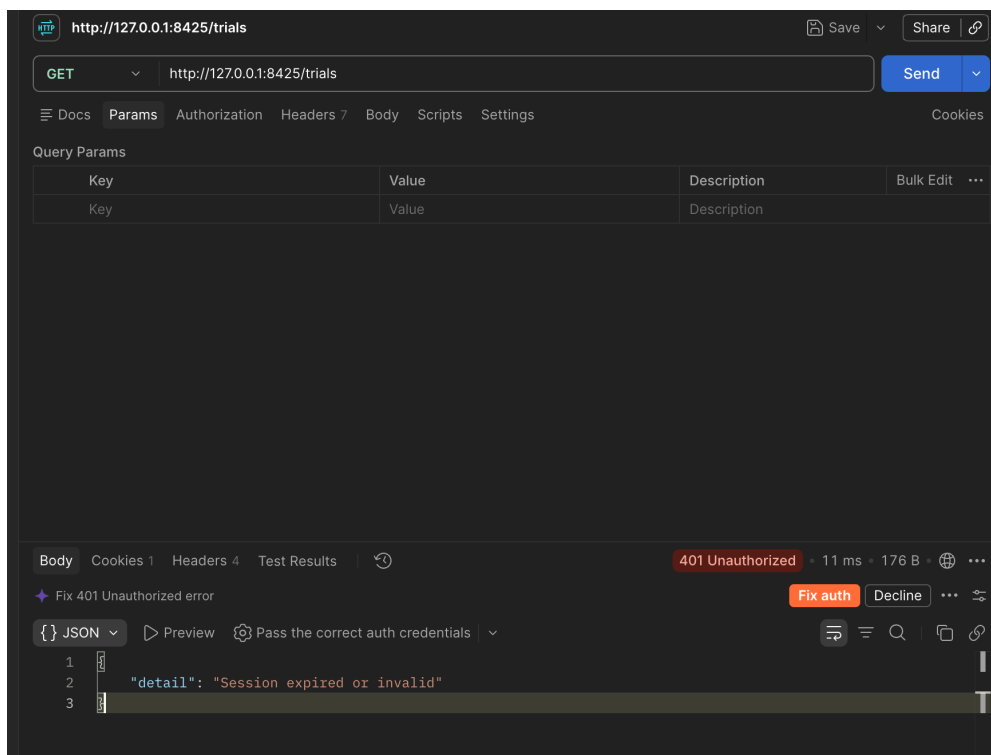
Visualize

```
1  [
2    {
3      "id": 1,
4      "trial_title": "Metformin Study",
5      "nct_number": "NCT04567890"
6    },
7    {
8      "id": 2,
9      "trial_title": "Aspirin Cardioprotective Study",
10     "nct_number": "NCT04987654"
11   }
12 ]
```

When the trial session ends after 30mins the GET method when called sends 401 unauthorized showing details as session expired or invalid

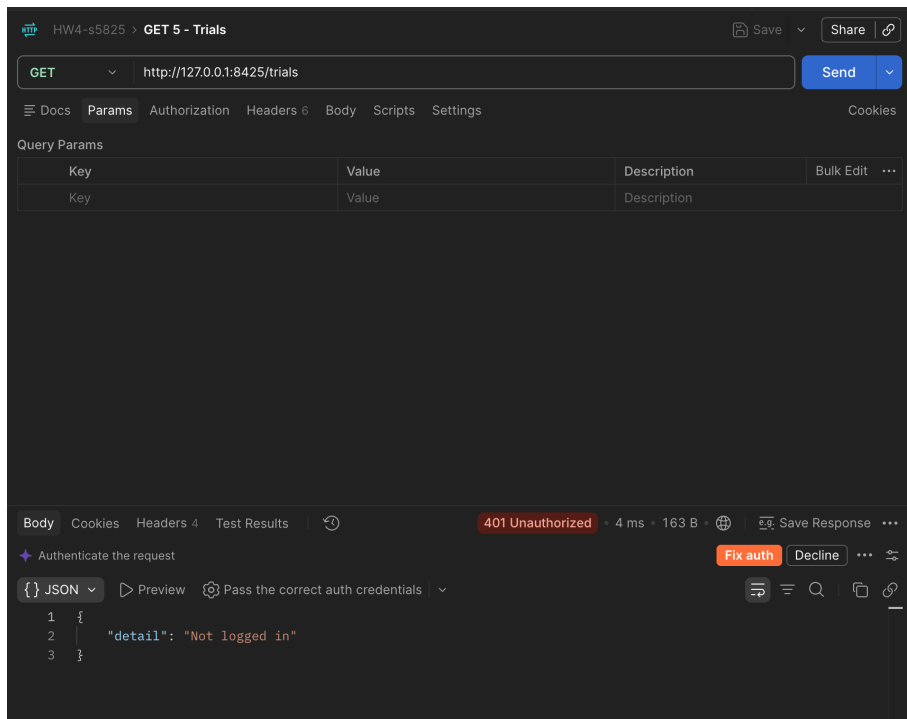
```
# ----- Gatekeeper: blocks requests without a valid session -----
def require_session(request: Request, db: Session = Depends(get_db)):
    token = request.cookies.get("session_id")
    if not token:
        raise HTTPException(status_code=401, detail="Not logged in")
    s = crud.get_session(db, token)
    if not s:
        raise HTTPException(status_code=401, detail="Session expired or invalid")
    return s
```

```
def get_session(db: Session, token: str):
    s = db.query(models.SessionToken).filter(models.SessionToken.id == token).first()
    if not s:
        return None
    if s.expires_at < datetime.utcnow():
        db.delete(s)
        db.commit()
        return None
    return s
```



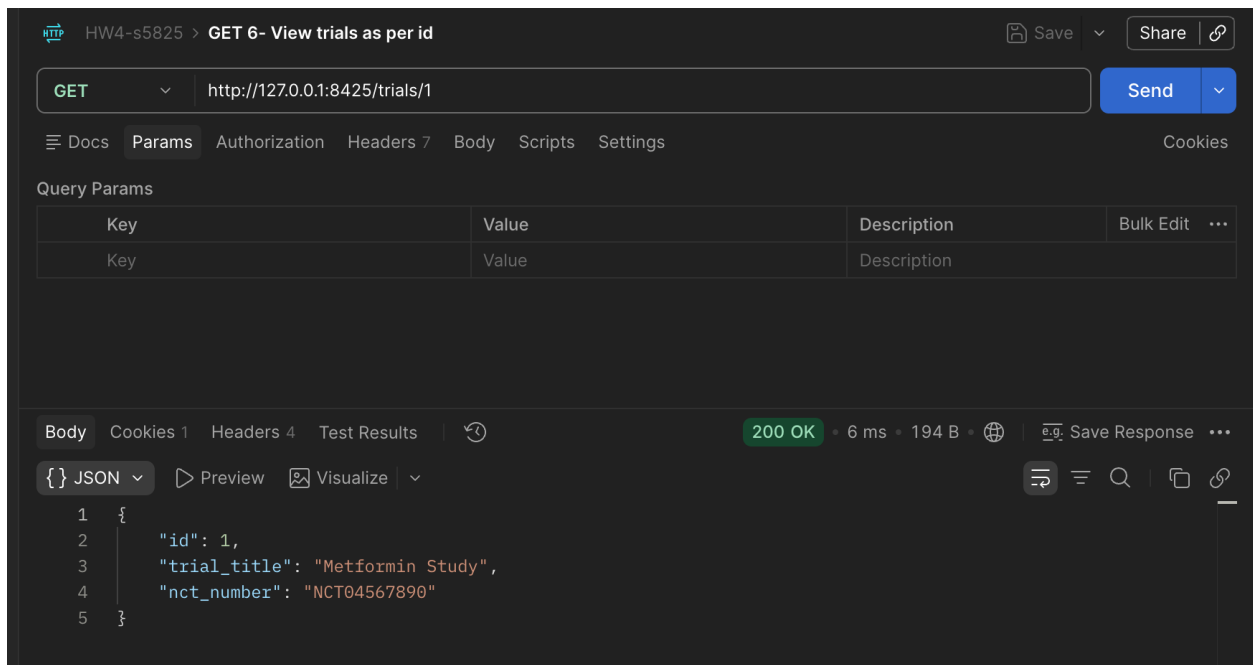
The screenshot shows a web browser interface with the URL `http://127.0.0.1:8425/trials`. The browser displays a 401 Unauthorized error. The error message is "401 Unauthorized" with a status of 11 ms and 176 B. The response body is JSON: `{"detail": "Session expired or invalid"}`. The browser interface includes tabs for "Body", "Cookies", "Headers", and "Test Results". The "Body" tab is selected, showing the JSON response. The "Cookies" tab shows no cookies. The "Headers" tab shows headers. The "Test Results" tab shows test results. The "Fix 401 Unauthorized error" button is visible, along with "Fix auth" and "Decline" buttons.

When the cookies are not stored that is user not logged in and still if we call for the trials list then it sends message as not logged in.



Request 3 – View record as per ID.

<http://127.0.0.1:8425/trials/1>



ID 2 - <http://127.0.0.1:8425/trials/2>

HTTP HW4-s5825 > GET 6- View trials as per id Save Share

GET ▼ http://127.0.0.1:8425/trials/2 Send ▼

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 200 OK 5 ms 209 B Save Response ...

{} JSON ▼ Preview Visualize ▼

```

1 {
2   "id": 2,
3   "trial_title": "Aspirin Cardioprotective Study",
4   "nct_number": "NCT04987654"
5 }
```

Code:

```

2
3 @app.get("/trials/{trial_id}", response_model=schema.TrialOut)
4 def get_trial(trial_id: int, db: Session = Depends(get_db), _s=Depends(require_session)):
5     trial = crud.get_trial(db, trial_id)
6     if not trial:
7         raise HTTPException(status_code=404, detail="Trial not found")
8     ✨ return trial
9

```

Request 4 – Update the record details:

Before update the ID =1 details are as follows:

```

1 {
2   "id": 1,
3   "trial_title": "Metformin Study",
4   "nct_number": "NCT04567890"
5 }
```

After update ID=1 details

The screenshot shows a REST client interface. The top bar indicates the URL is `http://127.0.0.1:8425/trials/1` and the method is `PUT`. The `Body` tab is selected, showing a raw JSON payload: `{"trial_title": "Metformin Study v2", "nct_number": "NCT04567890"}`. Below the request, the `Response` tab is selected, showing a `200 OK` status with a response time of `10 ms` and a size of `197 B`. The response body is a JSON object: `{ "id": 1, "trial_title": "Metformin Study v2", "nct_number": "NCT04567890" }`.

Code:

```
@app.put("/trials/{trial_id}", response_model=schema.TrialOut)
def edit_trial(trial_id: int, payload: schema.TrialUpdate, db: Session = Depends(get_db),
              _s=Depends(require_session)):
    trial = crud.update_trial(db, trial_id, payload)
    if not trial:
        raise HTTPException(status_code=404, detail="Trial not found")
    return trial
```

This request replaces the details of trial 1. `crud.update_trial` finds the row, overwrites the title and NCT number, and saves the change in MySQL. The same id is returned with the new values.

Request 5: Delete the trial

Deleting id=2 trial using delete method

Body Cookies 1 Headers 4 Test Results 200 OK • 6 ms • 28

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "trial_title": "Metformin Study v2",
5     "nct_number": "NCT04567890"
6   },
7   {
8     "id": 2,
9     "trial_title": "Aspirin Cardioprotective Study",
10    "nct_number": "NCT04987654"
11  }
12 ]
```

HW4-s5825 > DELETE - 8: Delete the trial

DELETE http://127.0.0.1:8425/trials/2

Docs Params Authorization Headers 7 Body Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies 1 Headers 4 Test Results 200 OK • 10 ms • 209

{ } JSON Preview Visualize

```
1 {
2   "id": 2,
3   "trial_title": "Aspirin Cardioprotective Study",
4   "nct_number": "NCT04987654"
5 }
```

Validated the same by calling GET method with id =2 that shows 404 error as trial not found

HTTP HW4-s5825 > GET 6- View trials as per id

GET http://127.0.0.1:8425/trials/2

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies 1 Headers 4 Test Results 404 Not Found • 6 ms • 162 B • e.g. Save Response ...

Investigate missing trial resource Diagnose Decline ...

{ } JSON Preview Debug with AI

```

1 {
2   "detail": "Trial not found"
3 }

```

Code:

```

9
0 @app.delete("/trials/{trial_id}", response_model=schema.TrialOut)
1 def remove_trial(trial_id: int, db: Session = Depends(get_db), _s=Depends(require_session)):
2     trial = crud.delete_trial(db, trial_id)
3     if not trial:
4         raise HTTPException(status_code=404, detail="Trial not found")
5     return trial

```

crud.delete_trial deletes the row from the trials table and returns what was deleted.

After deletion the db also gets updated now only 1 ID is visible in database.

id	trial_title	nct_number
1	Metformin Study v2	NCT04567890

The database also shows the user table with password as bcrypt hash. The password is not stored as plain text. In session table there are two rows, each with a token (only the first 12 characters are shown) and an expires_at. That's the server-side session storage. Two rows exist because I logged in twice.

```

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.trials;"
Enter password:
+-----+-----+-----+
| id | trial_title | nct_number |
+-----+-----+-----+
| 1 | Metformin Study | NCT04567890 |
+-----+-----+-----+

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.trials; SELECT id, name, email, password_hash FROM s5825_rel.users; SELECT LEFT(id,12) AS token, user_id, expires_at FROM s5825_rel.sessions;"
Enter password:
+-----+-----+-----+
| id | trial_title | nct_number |
+-----+-----+-----+
| 1 | Metformin Study v2 | NCT04567890 |
+-----+-----+-----+

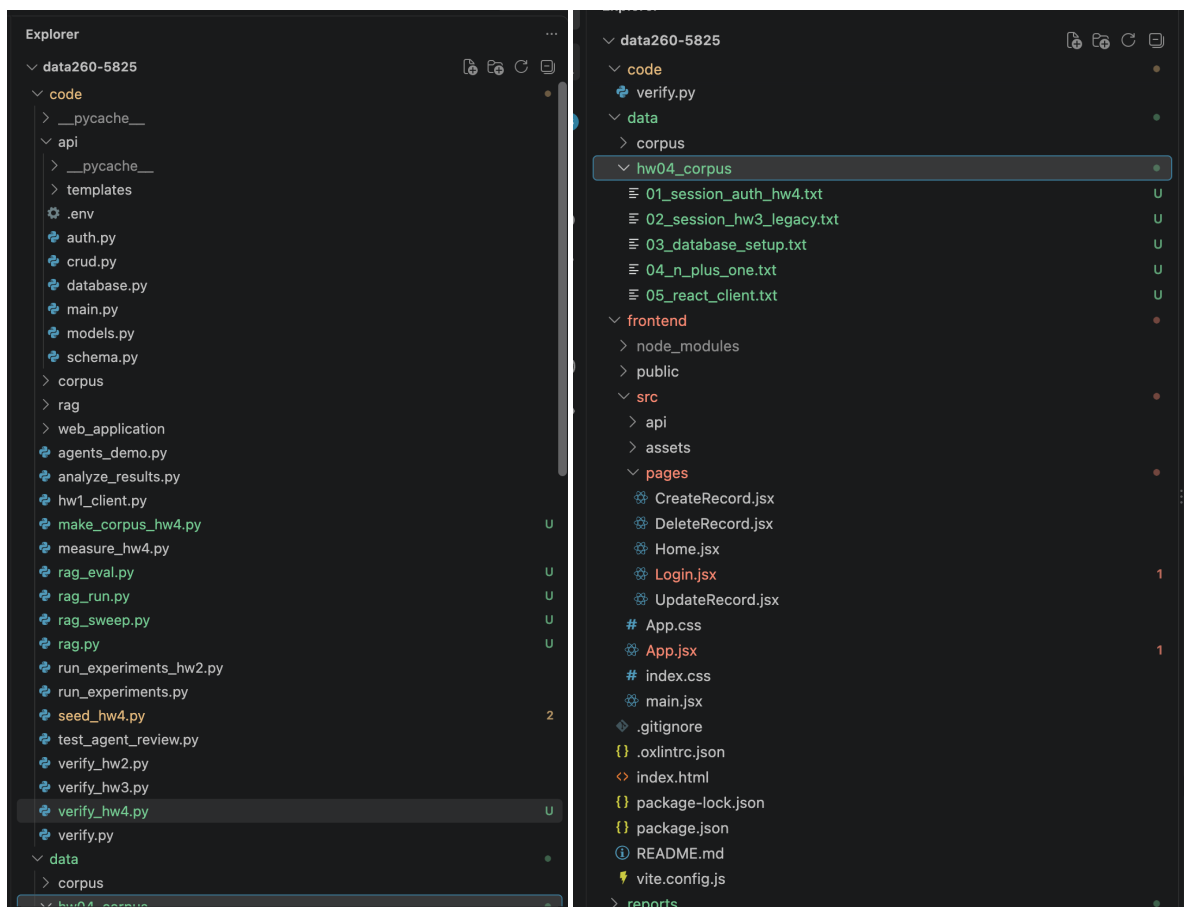
+-----+-----+-----+
| id | name | email | password_hash |
+-----+-----+-----+
| 1 | User1 | user1@gmail.com | $2b$12$Z2XPtLrS5pz1M1c9C5pu0eoR/90j0PjsxVNB6AwN53BtUIXSYDwa |
+-----+-----+-----+

+-----+-----+-----+
| token | user_id | expires_at |
+-----+-----+-----+
| 83076fc504d9 | 1 | 2026-09-28 04:58:28 |
| f239d1a49fa4 | 1 | 2026-09-28 04:58:20 |
+-----+-----+-----+

○ (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

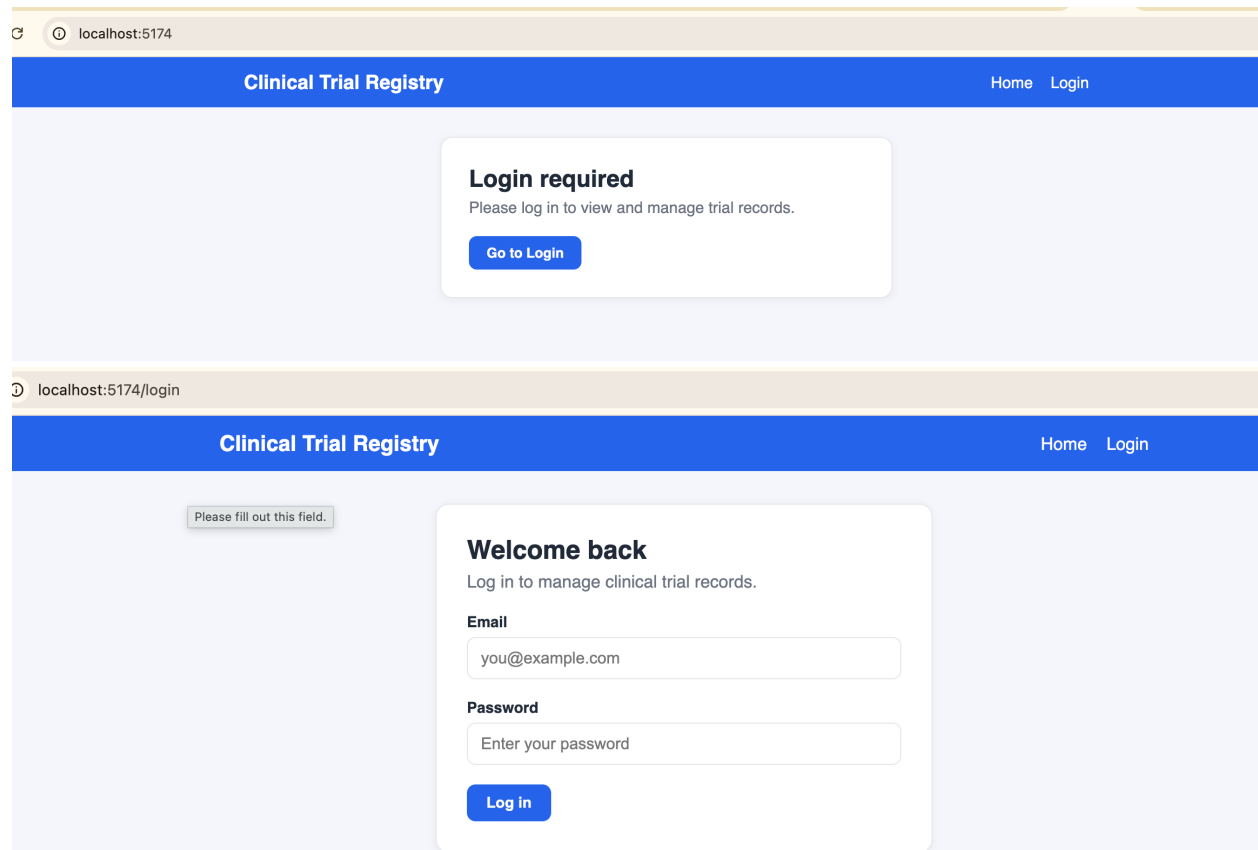
Project folder structure: backend files in code/api/ and frontend files in frontend/src/.



PART 1:

Two terminals – one running backend (uvicorn) and second running frontend (React)

Home page when not logged in (Home.jsx) shows "Login required" and hides the Add Trial link.



Logged in using the user credentials: user1@gmail.com and password for user1

localhost:5174

Clinical Trial Registry

HomeAdd TrialLogout

Clinical Trials

+ Add Trial

ID	TRIAL TITLE	NCT NUMBER	ACTIONS
1	Sitagliptin for Stroke Prevention #1	NCT10000000	EditDelete
2	Metformin for Kidney Disease #2	NCT10000001	EditDelete
3	Metformin for Kidney Disease #3	NCT10000002	EditDelete
4	Aspirin for Hypertension #4	NCT10000003	EditDelete
5	Sitagliptin for Obesity #5	NCT10000004	EditDelete
6	Empagliflozin for Type 2 Diabetes #6	NCT10000005	EditDelete
7	Lisinopril for Kidney Disease #7	NCT10000006	EditDelete
8	Aspirin for Heart Failure #8	NCT10000007	EditDelete
9	Empagliflozin for Kidney Disease #9	NCT10000008	EditDelete
10	Empagliflozin for High Cholesterol #10	NCT10000009	EditDelete
11	Metformin for Stroke Prevention #11	NCT10000010	EditDelete
12	Sitagliptin for High Cholesterol #12	NCT10000011	EditDelete

Added the trials: Add Trial form (CreateRecord.jsx) where the form is used to add a new clinical trial record.

localhost:5174/create

Clinical Trial Registry

HomeAdd TrialLogout

Add Clinical Trial

Enter the details of the new trial.

Trial Title

Aspirin Cardioprotective Study

NCT Number

NCT05123457

Add Trial

Cancel

localhost:5174

Clinical Trial Registry

HomeAdd TrialLogout

Clinical Trials

+ Add Trial

ID	TRIAL TITLE	NCT NUMBER	ACTIONS
1	Metformin Study v2	NCT04567890	EditDelete
3	Aspirin Cardioprotective Study	NCT04987654	EditDelete
4	Atorvastatin Cholesterol Trial	NCT05123456	EditDelete

Updated trial 3 to V2- Aspirin Cardioprotective Study and NCT no by clicking on Edit button

localhost:5174/update/3

Clinical Trial Registry

HomeAdd TrialLogout

Update Trial #3

Change the details below and save.

Trial Title

Aspirin Cardioprotective Study

NCT Number

NCT04987654

Update Trial

Cancel

localhost:5174/update/3

Clinical Trial Registry

HomeAdd TrialLogout

Update Trial #3

Change the details below and save.

Trial Title

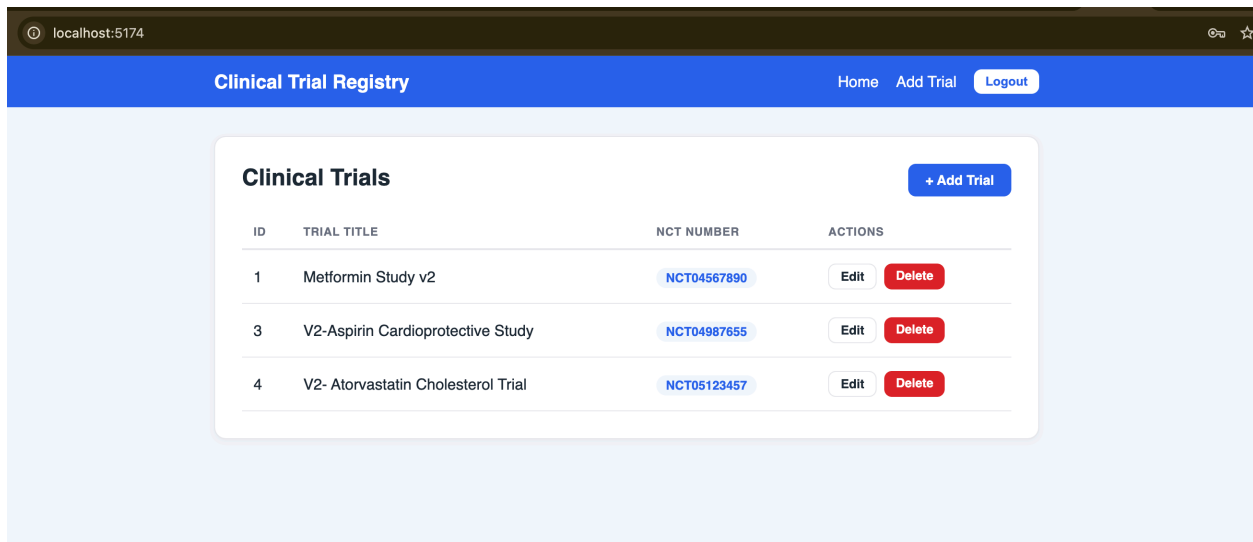
V2-Aspirin Cardioprotective Study

NCT Number

NCT04987655

Update Trial

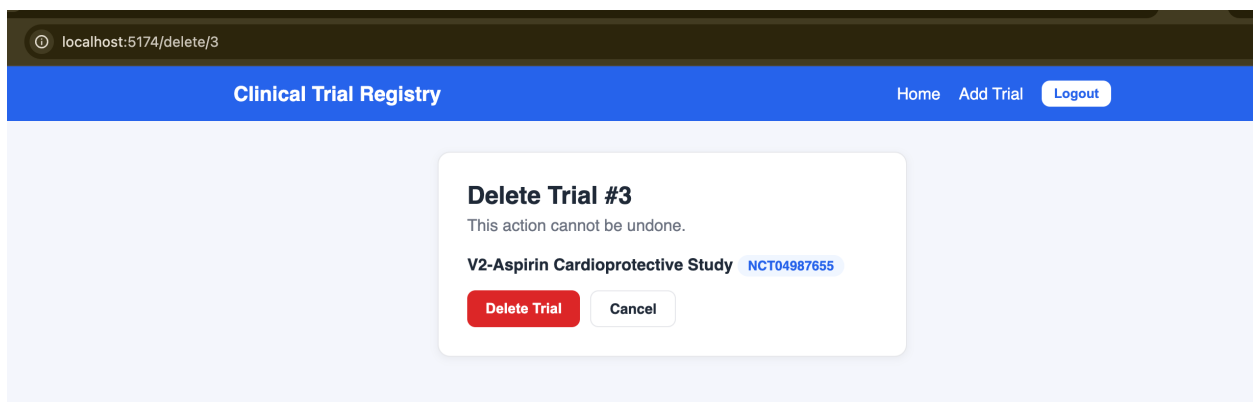
Cancel



Validated in database:

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.trials;"
Enter password:
+----+-----+-----+
| id | trial_title | nct_number |
+----+-----+-----+
| 1  | Metformin Study v2 | NCT04567890 |
| 3  | V2-Aspirin Cardioprotective Study | NCT04987655 |
| 4  | V2- Atorvastatin Cholesterol Trial | NCT05123457 |
+----+-----+-----+
```

Deleting one id that is ID=3 by clicking on delete button



List gets updated in db and in trials list

localhost:5174

Clinical Trial Registry Home Add Trial Logout

Clinical Trials

+ Add Trial

ID	TRIAL TITLE	NCT NUMBER	ACTIONS
1	Metformin Study v2	NCT04567890	Edit Delete
4	V2- Atorvastatin Cholesterol Trial	NCT05123457	Edit Delete
5	Semaglutide Weight Management Study	NCT05234567	Edit Delete

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT * FROM s5825_rel.trials;"
Enter password:
+-----+-----+-----+
| id | trial_title | nct_number |
+-----+-----+-----+
| 1 | Metformin Study v2 | NCT04567890 |
| 4 | V2- Atorvastatin Cholesterol Trial | NCT05123457 |
| 5 | Semaglutide Weight Management Study | NCT05234567 |
+-----+-----+-----+
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

Ln 8, Col 32 Sp

changed the session time to 30 secs so after login with the credentials it gets logged out automatically after 30 secs for testing but the cookies session id is still there, browser still holds the cookie (its max_age is 30 minutes), but the server-side session is already expired.

```
8 import schema
9
10 SESSION_TTL_MINUTES = 0.5
11
12
```

```
# session cookie support
app.add_middleware(
    SessionMiddleware,
    secret_key="s5825-dev-secret-change-later",
    session_cookie="s5825_session",
    max_age=300,
    https_only=True,
)
app.include_router(auth_router)
```

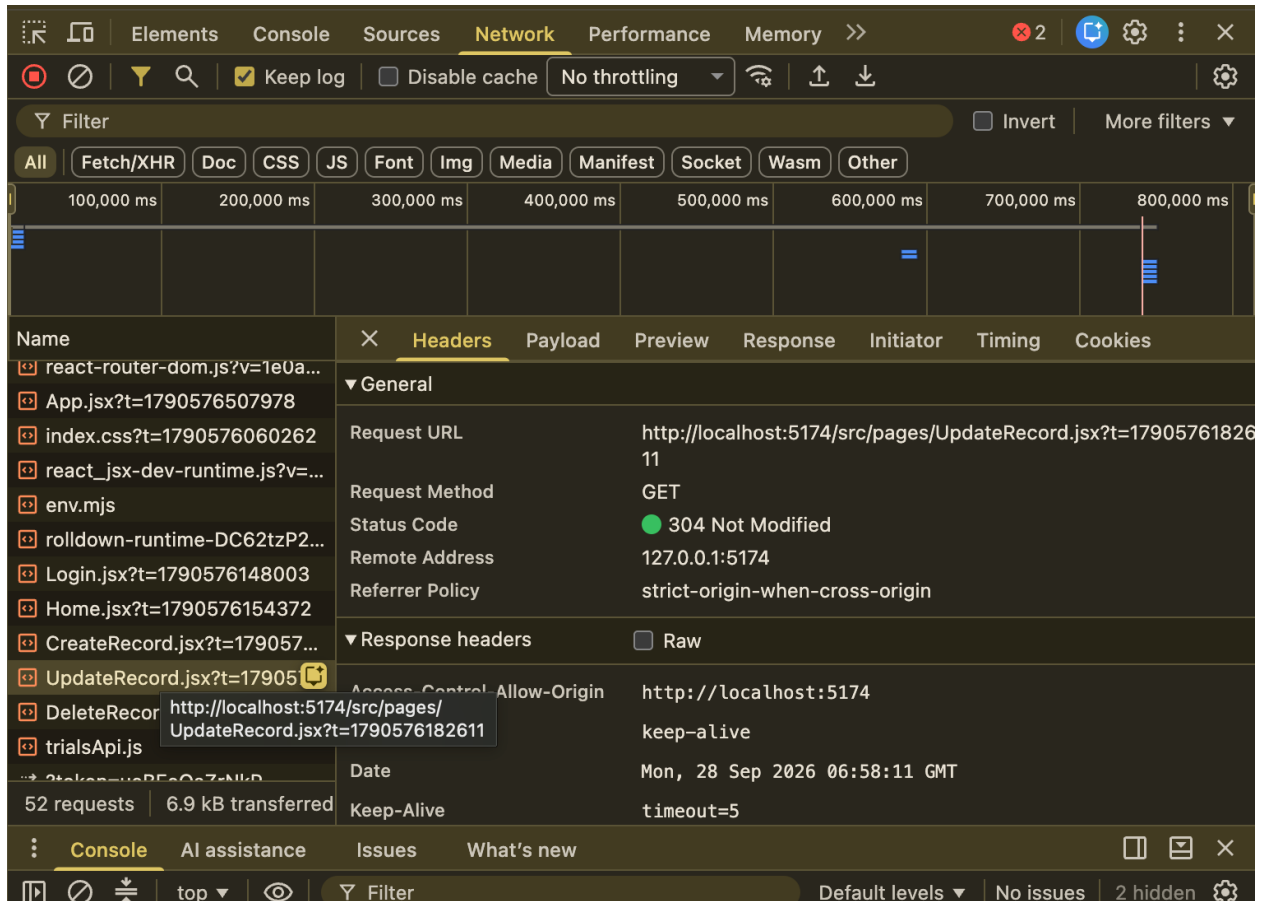
The screenshot shows the Chrome DevTools Application tab. The left sidebar has a tree view with categories: Application, Storage, Background..., and Console. Under Application, there are icons for Manifest, Service Worker, Storage, Variables (NEW), and Ads. Under Storage, there are icons for Local Storage, Session Storage, IndexedDB, and Cookies. The Cookies section is expanded, showing a single cookie for 'http://localhost:3000/'. The cookie details panel shows the 'Cookie Value' as '0f376500baee1dcdaa612fde1f43ff0a60c8ff6cab0185622b878825240c87eb'.

Name	Value	Domain	Path	Expires	Size	HttpOnly	Secure	SameSite	Partitioned	Creation	Priority
session_id	0f376500baee1dcdaa612f...	loc...	/	20...	74	✓		Lax			Me...

Cookie Value ☐ Show URL-decoded

0f376500baee1dcdaa612fde1f43ff0a60c8ff6cab0185622b878825240c87eb

validated the session cookie



Part 3 - N+1 Measurement and Query Tuning:

5,000 trials, 200 site rows, and each site links to a real trial through trial_id:

```

code > seed_hw4.py > ...
11 SEED = 5825
12 N_TRIALS = 5000
13 N_SITES = 200
14
15 rng = random.Random(SEED)
16
17 DRUGS = ["Metformin", "Aspirin", "Atorvastatin", "Semaglutide", "Insulin", "Lisinopril",
18         "Sitagliptin", "Empagliflozin", "Losartan", "Vildagliptin"]
19 CONDITIONS = ["Type 2 Diabetes", "Hypertension", "High Cholesterol", "Heart Failure",
20              "Obesity", "Kidney Disease", "Stroke Prevention"]
21 CITIES = ["San Jose", "Boston", "Chicago", "Houston", "Seattle", "Denver", "Atlanta", "Miami"]
22 HOSPITALS = ["General Hospital", "University Medical Center", "Research Institute", "Community Clinic"]
23
24 Base.metadata.create_all(bind=db_session_base26)
25 db = SessionLocal()
26
27 # start clean so the same SEED always gives the same data
28 db.execute(text("DELETE FROM trial_sites"))
29 db.execute(text("DELETE FROM trials"))
30 db.execute(text("ALTER TABLE trials AUTO_INCREMENT = 1"))
31 db.commit()
32
33 trials = []
34 for i in range(N_TRIALS):
35     title = f"{rng.choice(DRUGS)} for {rng.choice(CONDITIONS)} #{i + 1}"
36     trials.append(models.Trial(trial_title=title, nct_number=f"NCT10000000 + i"))
37 db.add_all(trials)
38 db.commit()
39
40 ids = [t.id for t in trials]
41 chosen = rng.sample(ids, N_SITES)
42 sites = []
43     models.TrialSite(
44         trial_id=tid,
45         site_name=f"{rng.choice(CITIES)} {rng.choice(HOSPITALS)}",
46         city=rng.choice(CITIES),
47     )
48     for tid in chosen
49 ]
50 db.add_all(sites)
51 db.commit()
52
53 print("trials:", db.query(models.Trial).count())
54 print("trial_sites:", db.query(models.TrialSite).count())
55 db.close()

```

```

(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p -e "SELECT COUNT(*) FROM s5825_rel.trials; SELECT COUNT(*) FROM s5825_re
l.trial_sites; SELECT * FROM s5825_rel.trial_sites LIMIT 3;"
Enter password:
+-----+
| COUNT(*) |
+-----+
| 5000 |
+-----+

+-----+
| COUNT(*) |
+-----+
| 200 |
+-----+

+----+-----+-----+-----+
| id | trial_id | site_name | city |
+----+-----+-----+-----+
| 201 | 4732 | Boston Research Institute | Boston |
| 202 | 3590 | Miami Research Institute | Miami |
| 203 | 1185 | Atlanta Community Clinic | Chicago |
+----+-----+-----+-----+
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

```

class TrialSite(Base):
    __tablename__ = "trial_sites"

    id = Column(Integer, primary_key=True, autoincrement=True)
    trial_id = Column(Integer, ForeignKey("trials.id", ondelete="CASCADE"), nullable=False)
    site_name = Column(String(255), nullable=False)
    city = Column(String(100), nullable=False)

```

```
# ----- Part 3: N+1 naive vs fixed list endpoints -----
@app.get("/trials-naive", response_model=list[schema.TrialWithSitesOut])
def trials_naive(response: Response, limit: int = 10, db: Session = Depends(get_db),
                 _s=Depends(require_session)):
    start = SQL_COUNT[0]
    trials = db.query(models.Trial).order_by(models.Trial.id).limit(limit).all() # 1 query
    result = []
    for t in trials:
        sites = db.query(models.TrialSite).filter(models.TrialSite.trial_id == t.id).all() # +1 per record
        result.append({
            "id": t.id, "trial_title": t.trial_title, "nct_number": t.nct_number,
            "sites": [{"id": s.id, "site_name": s.site_name, "city": s.city} for s in sites],
        })
    response.headers["X-SQL-Count"] = str(SQL_COUNT[0] - start)
    return result
```

HTTP <http://127.0.0.1:8425/trials-naive> Save Share

GET <http://127.0.0.1:8425/trials-naive?limit=10> Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ limit	10			
Key	Value	Description		

Body Cookies 1 Headers 5 Test Results 200 OK • 16 ms • 1.08 KB • ⌕ ...

Key	Value
Date	Mon, 28 Sep 2026 08:18:25 GMT
Server	uvicorn
Content-Length	962
Content-Type	application/json
X-Sql-Count	11

```
@app.get("/trials-fixed", response_model=list[schema.TrialWithSitesOut])
def trials_fixed(response: Response, limit: int = 10, db: Session = Depends(get_db),
                 _s=Depends(require_session)):
    start = SQL_COUNT[0]
    trials = (db.query(models.Trial)
              .options(joinedload(models.Trial.sites)) # one JOIN query
              .order_by(models.Trial.id).limit(limit).all())
    response.headers["X-SQL-Count"] = str(SQL_COUNT[0] - start)
    return trials
```

http://127.0.0.1:8425/trials-fixed

GET http://127.0.0.1:8425/trials-fixed?limit=10

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
limit	10		
Key	Value	Description	

Body Cookies 1 Headers 5 Test Results 200 OK • 16 ms • 1.08 KB

Key	Value
Date	Mon, 28 Sep 2026 08:18:45 GMT
Server	uvicorn
Content-Length	962
Content-Type	application/json
X-Sql-Count	1

Point 1: Schema file (reports/hw04/schema.sql) with all four tables.

```
(env) rutujasaste@Rutujas-MacBook-Pro data260-5825 % grep "CREATE TABLE" reports/hw04/schema.sql
CREATE TABLE `sessions` (
CREATE TABLE `trial_sites` (
CREATE TABLE `trials` (
CREATE TABLE `users` (
(env) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

Point 4: Asked for 50 trials using the slow version. It ran 51 database queries: 1 to get the list, plus 1 more for each of the 50 trials.

HW4-s5825 > GET: Asked for 50 trials

GET http://127.0.0.1:8425/trials-naive?limit=50

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
limit	50		
Key	Value	Description	

Body Cookies 1 Headers 5 Test Results 200 OK • 26 ms • 5.05 KB Save Response

JSON Preview Visualize

```
287   }
288   ],
289   {
290     "id": 48,
291     "trial_title": "Atorvastatin for Kidney Disease #48",
292     "nct_number": "NCT10000047",
293     "sites": [
294       {
295         "id": 298,
296         "site_name": "San Jose General Hospital",
297         "city": "Denver"
298       }
299     ]
300   },
301   {
302     "id": 49,
303     "trial_title": "Empagliflozin for Hypertension #49",
304     "nct_number": "NCT10000048",
305     "sites": []
306   },
307   {
308     "id": 50,
309     "trial_title": "Lisinopril for High Cholesterol #50",
310     "nct_number": "NCT10000049",
311     "sites": []
312   }
313 ]
314 ]
```

HW4-s5825 > GET: Asked for 50 trials

GET http://127.0.0.1:8425/trials-naive?limit=50

Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
limit	50		
Key	Value	Description	

Body Cookies 1 Headers 5 Test Results 200 OK • 26 ms • 5.05 KB Save Response

Key	Value
Date	Mon, 28 Sep 2026 17:08:10 GMT
Server	uvicorn
Content-Length	5021
Content-Type	application/json
X-Sql-Count	51

Request 2: Asked for the same 50 trials using the fast version. It ran only 1 database query, because the trials and their sites are loaded together.

GET	http://127.0.0.1:8425/trials-fixed?limit=50	Send
Docs	Params	Authorization
Headers	7	Body
Scripts	Settings	
Cookies		
Query Params		
Key	Value	Description
limit	50	
Key	Value	Description

Body	Cookies 1	Headers 5	Test Results	200 OK	6 ms	5.04 KB	Save Response
{ } JSON Preview Visualize							
<pre>289 }, 290 { 291 "id": 48, 292 "trial_title": "Atorvastatin for Kidney Disease #48", 293 "nct_number": "NCT10000047", 294 "sites": [295 { 296 "id": 298, 297 "site_name": "San Jose General Hospital", 298 "city": "Denver" 299 } 300] 301 }, 302 { 303 "id": 49, 304 "trial_title": "Empagliflozin for Hypertension #49", 305 "nct_number": "NCT10000048", 306 "sites": [] 307 }, 308 { 309 "id": 50, 310 "trial_title": "Lisinopril for High Cholesterol #50", 311 "nct_number": "NCT10000049", 312 "sites": [] 313 } 314]</pre>							

Standing by for launch

Body	Cookies 1	Headers 5	Test Results	200 OK	6 ms	5.04 KB	Save Response
Key Value							
Date	Mon, 28 Sep 2026 17:10:58 GMT						
Server	unicorn						
Content-Length	5021						
Content-Type	application/json						
X-Sql-Count	1						

Request 3: Asked for 200 trials using the slow version. It ran 201 database queries: 1 for the list plus 1 for each of the 200 trials.

HW4-s5825 > GET - Naive size =200

SaveShare

GEThttp://127.0.0.1:8425/trials-naive?limit=200Send

DocsParamsAuthorizationHeaders 7BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit	...
limit	200			
Key	Value	Description		

BodyCookies 1Headers 5Test Results200 OK61 ms19.79 KBSave Response

JSONPreviewVisualize

```
1211     "nct_number": "NCT10000195",
1212     "sites": []
1213   },
1214   {
1215     "id": 197,
1216     "trial_title": "Vildagliptin for Heart Failure #197",
1217     "nct_number": "NCT10000196",
1218     "sites": []
1219   },
1220   {
1221     "id": 198,
1222     "trial_title": "Aspirin for Hypertension #198",
1223     "nct_number": "NCT10000197",
1224     "sites": []
1225   },
1226   {
1227     "id": 199,
1228     "trial_title": "Losartan for Hypertension #199",
1229     "nct_number": "NCT10000198",
1230     "sites": []
1231   },
1232   {
1233     "id": 200,
1234     "trial_title": "Sitagliptin for Heart Failure #200",
1235     "nct_number": "NCT10000199",
1236     "sites": []
1237   }
1238 ]
```

Standing by for launch

BodyCookies 1Headers 5Test Results200 OK61 ms19.79 KBSave Response

Key	Value
Date	Mon, 28 Sep 2026 17:17:36 GMT
Server	uvicorn
Content-Length	20112
Content-Type	application/json
X-Sql-Count	201

Request 4: Asked for the same 200 trials using the fast version. It ran only 1 database query, no matter how many trials were requested.

HW4-s5825 > GET- fixed size 200

GET http://127.0.0.1:8425/trials-fixed?limit=200

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
limit	200			
Key	Value	Description		

Body Cookies 1 Headers 5 Test Results 200 OK • 11 ms • 19.78 KB Save Response

JSON Preview Visualize

```
1211 {"nct_number": "NCT10000195",
1212 "sites": []},
1213 {
1214   "id": 197,
1215   "trial_title": "Vildagliptin for Heart Failure #197",
1216   "nct_number": "NCT10000196",
1217   "sites": []
1218 },
1219 {
1220   "id": 198,
1221   "trial_title": "Aspirin for Hypertension #198",
1222   "nct_number": "NCT10000197",
1223   "sites": []
1224 },
1225 {
1226   "id": 199,
1227   "trial_title": "Losartan for Hypertension #199",
1228   "nct_number": "NCT10000198",
1229   "sites": []
1230 },
1231 {
1232   "id": 200,
1233   "trial_title": "Sitagliptin for Heart Failure #200",
1234   "nct_number": "NCT10000199",
1235   "sites": []
1236 },
1237 ]
1238 ]
```

Standing by for launch

Body Cookies 1 Headers 5 Test Results 200 OK • 11 ms • 19.78 KB Save Response

Key	Value
Date	Mon, 28 Sep 2026 17:19:55 GMT
Server	uvicorn
Content-Length	20112
Content-Type	application/json
X-Sql-Count	1

Point 6: Raw measurement files saved in reports/hw04/

```
Mon Sep 28 09:26:55 PDT 2026
• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % ls reports/hw04 reports/hw04/raw
reports/hw04:
METRICS.md      RUN_LOG.txt     raw
reports/hw04/raw:
requests_180.csv
○ (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

Preview of the csv file: The file has 180 data rows plus a header row

```

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % head -5 reports/hw04/raw/requests_180.csv
wc -l reports/hw04/raw/requests_180.csv
page_size,version,request_no,sql_stmts,latency_ms
10,naive,1,11,4.15
10,naive,2,11,3.81
10,naive,3,11,3.63
10,naive,4,11,3.82
181 reports/hw04/raw/requests_180.csv

```

All 180 measured requests (3 page sizes × 2 versions × 30 requests) were saved to reports/hw04/raw/requests_180.csv. The summary table is in METRICS.md, and the console output with timestamps is in RUN_LOG.txt. These files let anyone re-check the results.

Point 8: EXPLAIN before and after adding an index on trials.nct_number :

Before the index: Table scan on trials (cost=514 rows=5077)

```

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p s5825_rel -e "EXPLAIN SELECT * FROM trials WHERE nct_number='NCT10000042'"
Enter password:
+-----+
| EXPLAIN |
+-----+
| -> Filter: (trials.nct_number = 'NCT10000042') (cost=514 rows=508) |
| -> Table scan on trials (cost=514 rows=5077) |
+-----+
• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

After adding the index: Index lookup using idx_trials_nct_number(cost = 0.35,row =1)

```

• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % mysql -u root -p s5825_rel -e "EXPLAIN SELECT * FROM trials WHERE nct_number='NCT10000042'"
Enter password:
+-----+
| EXPLAIN |
+-----+
| -> Index lookup on trials using idx_trials_nct_number (nct_number = 'NCT10000042') (cost=0.35 rows=1) |
+-----+
• (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

Before the index, MySQL checked every row to find the NCT number. After adding the index, it goes directly to the matching row, making the search much faster and more efficient.

Query used: CREATE INDEX idx_trials_nct_number ON trials(nct_number); - This builds a lookup list on the nct_number column, so MySQL can jump straight to a value.

EXPLAIN SELECT * FROM trials WHERE nct_number='NCT10000042'; - This shows how MySQL plans to find the row without changing anything.

The updated implementation showed a clear performance improvement. For a page size of 10, it was about 2.3 times faster than the original. The improvement increased to 4.7 times for a page size of 50 and reached 10.3 times for a page size of 200. This happens because the original implementation makes an additional database request for every record it processes. As more records are included on a page, the number of database requests also increases, which slows down the response. The updated implementation avoids this

problem by retrieving the required records with one database query, regardless of the page size.

The difference is not as noticeable with smaller pages because other operations, such as handling the HTTP request and checking the user session, also contribute to the total response time. With larger pages, however, the repeated database requests become the main performance problem. Eliminating those extra requests is what allows the updated version to remain much faster.

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	11	3.78	4.08	4.15
10	fixed	1	1.67	1.77	1.79
50	naive	51	7.38	9.59	10.47
50	fixed	1	1.56	1.66	1.68
200	naive	201	25.13	28.3	33.4
200	fixed	1	2.43	2.65	8.66

Terminal output from running the measurement script, showing the same results as the table above with real timestamps.

```
(venv) rutujaste@Rutujast-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825
(date; python code/measure_hw4.py; date) | tee reports/hw04/RUN_LOG.txt
Mon Sep 28 09:26:53 PDT 2026
Page size | Version | SQL stmts/req | p50 (ms) | p95 (ms) | p99 (ms) |
-----|-----|-----|-----|-----|-----|
10 | naive | 11 | 3.78 | 4.08 | 4.15 |
10 | fixed | 1 | 1.67 | 1.77 | 1.79 |
50 | naive | 51 | 7.38 | 9.59 | 10.47 |
50 | fixed | 1 | 1.56 | 1.66 | 1.68 |
200 | naive | 201 | 25.13 | 28.3 | 33.4 |
200 | fixed | 1 | 2.43 | 2.65 | 8.66 |

Speed-up (naive p50 / fixed p50):
- size 10: 2.3x
- size 50: 4.7x
- size 200: 10.3x
Mon Sep 28 09:26:55 PDT 2026
(venv) rutujaste@Rutujast-MacBook-Pro data260-5825 %
```

PART 4:

1. Corpus and index:

Document collection and chunking (5 documents, chunk_size=500, chunk_overlap=50)

```

(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cd ~/Documents/data260-5825
python code/make_corpus_hw4.py
ls data/hw04_corpus
pip install faiss-cpu ollama numpy
ollama pull nomic-embed-text
ollama list
wrote 01_session_auth_hw4.txt 1096 characters
wrote 02_session_hw3_legacy.txt 832 characters
wrote 03_database_setup.txt 1082 characters
wrote 04_n_plus_one.txt 1609 characters
wrote 05_react_client.txt 956 characters
01_session_auth_hw4.txt      03_database_setup.txt      05_react_client.txt
02_session_hw3_legacy.txt    04_n_plus_one.txt
Requirement already satisfied: faiss-cpu in ./venv/lib/python3.11/site-packages (1.15.1)
Requirement already satisfied: ollama in ./venv/lib/python3.11/site-packages (0.6.2)
Requirement already satisfied: numpy in ./venv/lib/python3.11/site-packages (2.4.6)
Requirement already satisfied: packaging in ./venv/lib/python3.11/site-packages (from faiss-cpu) (26.3)
Requirement already satisfied: httpx>=0.27 in ./venv/lib/python3.11/site-packages (from ollama) (0.28.1)
Requirement already satisfied: pydantic>=2.9 in ./venv/lib/python3.11/site-packages (from ollama) (2.13.5)
Requirement already satisfied: anyio in ./venv/lib/python3.11/site-packages (from httpx>=0.27->ollama) (4.15.1)
Requirement already satisfied: certifi in ./venv/lib/python3.11/site-packages (from httpx>=0.27->ollama) (2026.7.22)
Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.11/site-packages (from httpx>=0.27->ollama) (1.0.9)
Requirement already satisfied: idna in ./venv/lib/python3.11/site-packages (from httpx>=0.27->ollama) (3.19)
Requirement already satisfied: h11>=0.16 in ./venv/lib/python3.11/site-packages (from httpcore==1.*->httpx>=0.27->ollama) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in ./venv/lib/python3.11/site-packages (from pydantic>=2.9->ollama) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in ./venv/lib/python3.11/site-packages (from pydantic>=2.9->ollama) (2.46.5)
Requirement already satisfied: typing-extensions>=4.14.1 in ./venv/lib/python3.11/site-packages (from pydantic>=2.9->ollama) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in ./venv/lib/python3.11/site-packages (from pydantic>=2.9->ollama) (0.4.4)
pulling manifest
pulling 970aa74c0a90: 100%
verifying sha256 digest
writing manifest
success
NAME                                ID                                SIZE    MODIFIED
nomic-embed-text:latest             0a109f422b47                     274 MB  Less than a second ago
qwen3:8b                             500a1f067a9f                     5.2 GB  2 weeks ago
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % python code/rag.py

```

The embedding model is nomic-embed-text and the answering model is the local qwen3:8b, both run through Ollama.

The five documents were split into 15 chunks.

```

(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % python code/rag.py
python code/rag.py "Which index was added and what did EXPLAIN show?"
documents: 5  chunks: 15  (chunk_size=500, overlap=50)

```

2. Retrieval printed before the model is called

```

2
3 def retrieve(index, chunks, question, k=3):
4     scores, ids = index.search(embed([question]), k)
5     return [dict(chunks[i], score=float(s)) for s, i in zip(scores[0], ids[0])]
6
7
8 def print_retrieval(question, hits):
9     print(f"\nQUESTION: {question}")
10    for rank, h in enumerate(hits, 1):
11        print(f"    [{rank}] score={h['score']:.4f} source={h['source']} chunk_id={h['chunk_id']}")
12        print("        " + h["text"][:200].replace("\n", " ") + "...")
13
14
15 QUESTION: How long does a HW4 session last?
16 [1] score=0.7595 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
17     d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on
18 the server. Logging out deletes the session row and clears the ...
19 [2] score=0.6627 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
20     Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data
21 is stored inside a signed cookie named s5825_session instead o...
22 [3] score=0.6506 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
23     Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The conne
24 ction object in database.py is named db_session_base26, and S...
25 documents: 5  chunks: 15  (chunk_size=500, overlap=50)

```

The top chunk comes from 03_database_setup.txt and holds the EXPLAIN before and after result.

```

QUESTION: Which index was added and what did EXPLAIN show?
[1] score=0.7286 source=03_database_setup.txt chunk_id=03_database_setup.txt#2
    Before the index, EXPLAIN showed a table scan with an estimated cost of 514 over about 5077 rows. After the index, EXPLAIN showed an index
    x lookup with an estimated cost of 0.35 and 1 row...
[2] score=0.5862 source=03_database_setup.txt chunk_id=03_database_setup.txt#1
    le holds the session token as its id, the user_id, created_at and expires_at. The trial_sites table holds test data, with an id, a trial_
    id that points to a trial, a site_name and a city. The databas...
[3] score=0.5336 source=04_n_plus_one.txt chunk_id=04_n_plus_one.txt#3
    or naive and 1.67 ms for fixed at size 10, 7.38 ms and 1.56 ms at size 50, and 25.13 ms and 2.43 ms at size 200. The speed-up was 2.3 tim
    es, 4.7 times and 10.3 times. The speed-up grows because every ...
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

Test set of six questions:

Three-configuration comparison, run 1 (grounding rules v1)

```

# ----- (A) No RAG -----
def config_a(q):
    return ask_llm(q), []

# ----- (B) Basic RAG: top-k raw chunks pasted in -----
def config_b(q, hits):
    context = "\n\n".join(h["text"] for h in hits)
    return ask_llm(f"Context:\n{context}\n\nQuestion: {q}\nAnswer:", hits)

# ----- (C) Context-engineered RAG -----
def words(t):
    return set(re.findall(r"\w+", t.lower()))

def engineer(hits):
    kept = []
    for h in hits:
        if h["score"] < MIN_SCORE:
            continue # drop irrelevant
        w = words(h["text"])
        if any(len(w & words(k["text"])) / len(w | words(k["text"])) > 0.8 for k in kept):
            continue # drop duplicate
        kept.append(h)
    return sorted(kept, key=lambda h: -h["score"]) # best evidence first

def config_c(q, hits):
    kept = engineer(hits)
    if not kept:
        return REFUSAL + ".", kept
    context = "\n\n".join(f"[{i}] {h['source']}\n{h['text']}"
                        for i, h in enumerate(kept, 1))
    prompt = (
        "You answer using ONLY the numbered context below.\n"
        "Rules:\n"
        "1. Use only facts written in the context. Do not use outside knowledge.\n"
        "2. After each fact, cite the source number like [1].\n"
        "3. If the context does not contain the answer, reply exactly: {REFUSAL}.\n"
        "4. If the question could mean more than one thing, say so and answer each meaning.\n\n"
        f"Context:\n{context}\n\nQuestion: {q}\nAnswer:"
    )
    return ask_llm(prompt), kept

```

Questions list:

```

QUESTIONS = [
    {"id": "Q1", "type": "one chunk",
     "question": "How long does a HW4 session last?",
     "expected": "30 minutes", "source": "01_session_auth_hw4.txt"},
    {"id": "Q2", "type": "two chunks",
     "question": "What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?",
     "expected": "session_id; 5 minutes", "source": "01_session_auth_hw4.txt + 02_session_hw3_legacy.txt"},
    {"id": "Q3", "type": "similar info across documents",
     "question": "How do the HW3 and HW4 sessions differ in where the session data is stored?",
     "expected": "HW3: inside a signed cookie; HW4: in the sessions table in MySQL, cookie holds only a token",
     "source": "01_session_auth_hw4.txt + 02_session_hw3_legacy.txt"},
    {"id": "Q4", "type": "ambiguous",

```

```
"question": "How long does the session last?",
"expected": "ambiguous: HW3 is 5 minutes, HW4 is 30 minutes",
"source": "01_session_auth_hw4.txt + 02_session_hw3_legacy.txt"},
{"id": "Q5", "type": "not in documents",
"question": "What was the p99 latency of the fixed endpoint at page size 200 in the second
measurement run?",
"expected": "REFUSE (p99 is not in the documents)", "source": "none"},
{"id": "Q6", "type": "unrelated",
"question": "What is the capital of France?",
"expected": "REFUSE", "source": "none"},
]
```

Six questions were tested using three methods:

- No RAG
- Basic RAG using the top three chunks
- Improved RAG that removes weak or repeated information and adds source labels and grounding rules

All methods handled Q1–Q3 and Q6 correctly. However, two issues occurred:

- For Q5, the system gave a p99 value of 2.43 ms, even though the documents only provided a median value.
- For Q4, the system gave the HW4 answer of 30 minutes but missed the HW3 answer of 5 minutes.

```

• (venv) rutujasast@Rutujas-MacBook-Pro data260-5825 % python code/rag_run.py | tee reports/hw04/raw/retrieval_and_comparison_log.txt

=====
Q1 (one chunk)

QUESTION: How long does a HW4 session last?
[1] score=0.7595 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
    d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on
the server. Logging out deletes the session row and clears the ...
[2] score=0.6627 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
    Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data
is stored inside a signed cookie named s5825_session instead o...
[3] score=0.6506 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
    Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connec
tion object in database.py is named db_session_based26, and S...

(A) No RAG : A "HW4 session" is not a standard term, and its meaning can vary depending on the context in which it is used. Here are a f
ew possible interpretations:

1. **Homework Session (HW4)**: If "HW4" refers to Homework 4 in a class or course, then the duration of a "session" could depend on the instruc
tor's schedule. It might be a single class period (e.g., 50-75 minutes), or it could refer to the time students spend working on the assignment
outside of class.

2. **Software or Application Session**: If "HW4" is part of a software or application (e.g., a homework platform or learning management system)
, the session duration might be defined by the system. For example, some platforms might have time limits for completing assignments or taking
quizzes.

3. **Custom or Institutional Definition**: In some educational institutions or online learning platforms, "HW4 session" might be a specific ter
m used internally, and the duration would be defined by the institution.

If you can provide more context (e.g., which course, platform, or institution you're referring to), I can give a more accurate answer.
(B) Basic RAG : A **HW4 session lasts 30 minutes**.

This is explicitly stated in the context: "A HW4 session lasts 30 minutes on the server."*

Additionally, logging out deletes the session row and clears the cookie, which aligns with the behavior of session management in the system.
(C) Context RAG : A HW4 session lasts 30 minutes on the server [1]. [kept 3 of 3 chunks]

=====
Q2 (two chunks)

QUESTION: What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?
[1] score=0.8052 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
    Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data
is stored inside a signed cookie named s5825_session instead o...
[2] score=0.7954 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
    d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on
the server. Logging out deletes the session row and clears the ...
[3] score=0.6506 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
    Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks
the password against a bcrypt hash stored in the users table. ...

(A) No RAG : The questions you've asked appear to be related to specific technical details of a web application or system, possibly from
a course or lab (like HW4 and HW3). However, without additional context such as the specific course, platform, or system being referred to, it
's not possible to provide exact answers.

If you're referring to a particular course or platform (e.g., a university course, a coding lab, or a specific web application), please provide
more details, such as:

- The name of the course or platform

=====
Q3 (similar info across documents)

QUESTION: How do the HW3 and HW4 sessions differ in where the session data is stored?
[1] score=0.7324 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
    Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connec
tion object in database.py is named db_session_based26, and S...
[2] score=0.7196 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
    Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data
is stored inside a signed cookie named s5825_session instead o...
[3] score=0.6675 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
    Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks
the password against a bcrypt hash stored in the users table. ...

(A) No RAG : In the context of web development, particularly when using frameworks like **Express.js** or **Node.js**, the **HW3** and *
**HW4** sessions (which likely refer to different homework assignments or versions of a project) may differ in how they store session data. Here
's a breakdown of how they might differ:

---

## ♦ **HW3: In-Memory Session Storage**

- **Storage Location**: Session data is stored **in memory** (i.e., in the server's RAM).
- **How It Works**:
    - The session data is stored on the **server side**.
    - Each session is stored in a **JavaScript object** in memory.
    - The session ID is sent to the client (usually via a cookie), and the server uses that ID to retrieve the session data.
- **Pros**:
    - Simple to implement.
    - No external dependencies.
- **Cons**:
    - Not scalable (data is lost if the server restarts).
    - Not suitable for distributed systems (multiple servers can't share the same in-memory store).
- **Example**:
    ```js
 app.use(session({
 secret: 'mysecretkey',
 resave: false,
 saveUninitialized: false,
 store: new MemoryStore() // or just default
 }));
    ```

---

## ♦ **HW4: Persistent Session Storage (e.g., Redis, MongoDB, etc.)**

- **Storage Location**: Session data is stored in a **persistent storage system** like **Redis**, **MongoDB**, or a **database**.
- **How It Works**:
    - The session data is stored **externally**, not in memory.
    - The server still uses a session ID (cookie) to reference the session, but the actual data is stored in a **database or cache**.
    - This allows for **scalability**, **session persistence**, and **load balancing** across multiple servers.
- **Pros**:
    - Persistent across server restarts.
    - Suitable for distributed systems.
    - Can be shared across multiple servers.
- **Cons**:

```

```

- Cons:
- Requires additional setup (e.g., Redis server, database).
- Slightly more complex to implement.
- Example:
```js
app.use(session({
 secret: 'mysecretkey',
 resave: false,
 saveUninitialized: false,
 store: new RedisStore({ host: 'localhost', port: 6379 }) // or MongoDB store
}));
```

Summary of Differences

Feature	HW3 (In-Memory)	HW4 (Persistent)
Storage Location	In server memory	In external storage (Redis, DB, etc.)
Persistence	Not persistent (lost on restart)	Persistent
Scalability	Not scalable	Scalable
Distributed Systems	Not suitable	Suitable
Complexity	Simple	More complex
Use Case	Small, local apps	Larger, distributed apps

If you're referring to a specific framework or course, let me know, and I can tailor the explanation further!
(B) Basic RAG : Answer:
In HW3, session data is stored in a signed cookie named `s5825_session` on the client side (browser), while in HW4, session data is stored in a database table (the `sessions` table) on the server side.
(C) Context RAG : The HW3 session data is stored in a signed cookie named s5825_session [2], while the HW4 session data is stored in a database table named sessions [1]. [kept 3 of 3 chunks]
=====
Q4 (ambiguous)

QUESTION: How long does the session last?
[1] score=0.6828 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[2] score=0.6347 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0

```

Q4: Ambiguous

```

se table named sessions [1]. [kept 3 of 3 chunks]
=====
Q4 (ambiguous)

QUESTION: How long does the session last?
[1] score=0.6828 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[2] score=0.6347 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
[3] score=0.6230 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...

(A) No RAG : The duration of a session can vary depending on the context. Here are a few common examples:

1. Online Meetings or Webinars: Typically last from 30 minutes to several hours, depending on the agenda.
2. Therapy or Counseling Sessions: Usually last 45 to 60 minutes.
3. Classroom Sessions: Can range from 45 minutes to 2 hours, depending on the institution and subject.
4. Workshops or Training Sessions: Often last 1 to 3 hours.
5. Video Calls: Can be as short as 10 minutes or as long as several hours.

If you're referring to a specific type of session (e.g., a therapy session, a Zoom meeting, a class), please provide more details so I can give you a more accurate answer.
(B) Basic RAG : The session lasts 30 minutes.

Explanation:
- The HW4 session lasts 30 minutes on the server.
- When a user logs in, a session token is created and stored in the `sessions` table with an expiry time set to 30 minutes from creation.
- If the session expires (after 30 minutes), the server returns a 401 Unauthorized response with the message "Session expired or invalid", and the expired session row is deleted from the database.
- Logging out explicitly deletes the session row and clears the cookie.

In contrast, HW3 uses a different session handling mechanism with a 5-minute idle timeout, but that is unrelated to the HW4 session duration.
(C) Context RAG : The session lasts for 30 minutes [1]. [kept 3 of 3 chunks]
=====
Q5 (not in documents)

```

```
=====
Q5 (not in documents)
QUESTION: What was the p99 latency of the fixed endpoint at page size 200 in the second measurement run?
[1] score=0.6640 source=04_n_plus_one.txt chunk_id=04_n_plus_one.txt#2
    easured requests. The number of SQL statements per request was read from the X-SQL-Count response header, which does not count the one se
ssion check query. Latency was measured with a timer around eac...
[2] score=0.6227 source=04_n_plus_one.txt chunk_id=04_n_plus_one.txt#3
    or naive and 1.67 ms for fixed at size 10, 7.38 ms and 1.56 ms at size 50, and 25.13 ms and 2.43 ms at size 200. The speed-up was 2.3 tim
es, 4.7 times and 10.3 times. The speed-up grows because every ...
[3] score=0.5792 source=03_database_setup.txt chunk_id=03_database_setup.txt#2
    Before the index, EXPLAIN showed a table scan with an estimated cost of 514 over about 5077 rows. After the index, EXPLAIN showed an inde
x lookup with an estimated cost of 0.35 and 1 row....

(A) No RAG      : To answer your question about the **p99 latency** of the fixed endpoint at **page size 200** in the **second measurement ru
n**, I would need access to the specific data or results from that measurement run.

Since I don't have access to external data or previous measurement runs, I can't provide the exact value. If you can share the relevant data or
results (e.g., a table, graph, or log), I'd be happy to help you extract the p99 latency for that specific case.
(B) Basic RAG   : The question asks for the **p99 latency of the fixed endpoint at page size 200 in the second measurement run**.

From the context:

- For **page size 200**, the **latency** for the **fixed endpoint** is given as **2.43 ms**.
- The **p99 latency** is **computed from the 30 values**, and in this case, it is **reported as 2.43 ms**.

Therefore, the **p99 latency of the fixed endpoint at page size 200** is:

**Answer: 2.43 ms**.
(C) Context RAG : The p99 latency of the fixed endpoint at page size 200 was 2.43 ms [2]. [kept 3 of 3 chunks]
=====
Q6 (unrelated)
QUESTION: What is the capital of France?
[1] score=0.3994 source=03_database_setup.txt chunk_id=03_database_setup.txt#1
    le holds the session token as its id, the user_id, created_at and expires_at. The trial_sites table holds test data, with an id, a trial_
id that points to a trial, a site_name and a city. The databas...
[2] score=0.3729 source=05_react_client.txt chunk_id=05_react_client.txt#0
    React client for the Clinical Trial Registry. The frontend is a React app built with Vite and React Router. It has five pages: Login.jsx,
Home.jsx, CreateRecord.jsx, UpdateRecord.jsx and DeleteRecord....
[3] score=0.3692 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
    Database setup for HW4. The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connec
tion object in database.py is named db_session_base26, and S...

(A) No RAG      : The capital of France is Paris.
(B) Basic RAG   : Paris.
(C) Context RAG : I cannot answer this question from the provided documents. [kept 0 of 3 chunks]

saved reports/hw04/raw/three_config_results.json
o (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

After change: Fix: stricter grounding rules

Final run: config C refuses Q5 and Q6 and answers Q1 to Q4 with source citations.

Prompt rules alone did not stop the small local model from treating a median as a p99, so a plain-Python check was added. It refuses when a metric named in the question (p99) never appears in the retrieved text.

```
(venv) rutujasate@Rutujas-MacBook-Pro data260-5825 % python code/rag_run.py | tee reports/hw04/raw/retrieval_and_comparison_log.txt
=====
Q1 (one chunk)
QUESTION: How long does a HW4 session last?
[1] score=0.7595 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session
row and clears the ...
[2] score=0.6627 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s58
25_session instead o...
[3] score=0.6506 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_ses
sion_base26, and S...

(A) No RAG : A "HW4 session" is not a standard term, and its meaning can vary depending on the context in which it is used. Here are a few possible interpretations:

1. **Homework Session (HW4)**: If "HW4" refers to Homework 4 in a class or course, then the duration of a "session" could depend on the instructor's schedule. It might be a single class
period (e.g., 50-75 minutes), or it could refer to the time students spend working on the assignment outside of class.

2. **Software or Application Session**: If "HW4" is part of a software or application (e.g., a homework platform or learning management system), the session duration might be defined by
the system. For example, some platforms might have time limits for completing assignments or taking quizzes.

3. **Custom or Institutional Definition**: In some educational institutions or online learning platforms, "HW4 session" might be a specific term used internally, and the duration would b
e defined by the institution.

If you can provide more context (e.g., which course, platform, or institution you're referring to), I can give a more accurate answer.
(B) Basic RAG : A HW4 session lasts 30 minutes**.

This is explicitly stated in the context: "A HW4 session lasts 30 minutes on the server."

Additionally, logging out deletes the session row and clears the cookie, which aligns with the behavior of session management in the system.
(C) Context RAG : A HW4 session lasts 30 minutes on the server [1]. [kept 3 of 3 chunks]
=====
Q2 (two chunks)
QUESTION: What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?
[1] score=0.8052 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s58
25_session instead o...
[2] score=0.7954 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session
row and clears the ...
[3] score=0.6530 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored
in the users table. ...

(A) No RAG : The questions you've asked appear to be related to specific technical details of a web application or system, possibly from a course or lab (like HW4 and HW3). Howeve
r, without additional context such as the specific course, platform, or system being referred to, it's not possible to provide exact answers.

If you're referring to a particular course or platform (e.g., a university course, a coding lab, or a specific web application), please provide more details, such as:

- The name of the course or platform
- Any relevant documentation or code snippets
- The context in which HW4 and HW3 are mentioned

With that information, I can help you identify the session cookie name and the idle timeout duration.
(B) Basic RAG : The name of the HW4 session cookie is **session_id**.
```

```
main 2
Problems Output Debug Console Terminal Ports
-----
The HW3 idle timeout is **5 minutes (300 seconds)**.
```

```
=====
(C) Context RAG : The name of the HW4 session cookie is session_id [3]. The HW3 idle timeout is 5 minutes [1]. [kept 3 of 3 chunks]
=====
Q3 (similar info across documents)
QUESTION: How do the HW3 and HW4 sessions differ in where the session data is stored?
[1] score=0.7324 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_ses
sion_base26, and S...
[2] score=0.7196 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s58
25_session instead o...
[3] score=0.6675 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored
in the users table. ...

(A) No RAG : In the context of web development, particularly when using frameworks like **Express.js** or **Node.js**, the **HW3** and **HW4** sessions (which likely refer to diff
erent homework assignments or versions of a project) may differ in how they store session data. Here's a breakdown of how they might differ:

---
### ♦ **HW3: In-Memory Session Storage**

- **Storage Location:** Session data is stored **in memory** (i.e., in the server's RAM).
- **How It Works:**
  - The session data is stored on the **server side**.
  - Each session is stored in a **JavaScript object** in memory.
  - The session ID is sent to the client (usually via a cookie), and the server uses that ID to retrieve the session data.
- **Pros:**
  - Simple to implement.
  - No external dependencies.
- **Cons:**
  - Not scalable (data is lost if the server restarts).
  - Not suitable for distributed systems (multiple servers can't share the same in-memory store).
- **Example:**
  ...js
  app.use(session({
    secret: 'mysecretkey',
    resave: false,
    saveUninitialized: false,
    store: new MemoryStore() // or just default
  }));

---
### ♦ **HW4: Persistent Session Storage (e.g., Redis, MongoDB, etc.)**

- **Storage Location:** Session data is stored in a **persistent storage system** like **Redis**, **MongoDB**, or a **database**.
- **How It Works:**
  - The session data is stored **externally**, not in memory.
  - The server still uses a session ID (cookie) to reference the session, but the actual data is stored in a **database or cache**.
  - This allows for **scalability**, **session persistence**, and **load balancing** across multiple servers.
- **Pros:**
  - Persistent across server restarts.
  - Suitable for distributed systems.
  - Can be shared across multiple servers.
```

```
- Requires additional setup (e.g., Redis server, database).
- Slightly more complex to implement.
- **Example:**
  """
  app.use(session({
    secret: 'mysecretkey',
    resave: false,
    saveinitialized: false,
    store: new RedisStore({ host: 'localhost', port: 6379 }) // or MongoDB store
  }));
  """

## ✅ **Summary of Differences**



Feature	HW3 (In-Memory)	HW4 (Persistent)
**Storage Location**	In server memory	In external storage (Redis, DB, etc.)
**Persistence**	Not persistent (lost on restart)	Persistent
**Scalability**	Not scalable	Scalable
**Distributed Systems**	Not suitable	Suitable
**Complexity**	Simple	More complex
**Use Case**	Small, local apps	Larger, distributed apps



If you're referring to a **specific framework or course**, let me know, and I can tailor the explanation further!

(B) Basic RAG : **Answer:**
In **HW3**, session data is stored in a **signed cookie** named `s5825_session` on the client side (browser), while in **HW4**, session data is stored in a **database table** (the `sessions` table) on the server side.
(C) Context RAG : HW3 stores session data in a signed cookie named `s5825_session` [2]. HW4 stores session data in a database table named sessions [1]. [kept 3 of 3 chunks]

=====
Q4 (ambiguous)
QUESTION: How long does the session last?
[1] score=0.6828 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[2] score=0.6547 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a User logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
[3] score=0.6238 source=02_session_hw2_legacy.txt chunk_id=02_session_hw2_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead of ...
(A) No RAG : The duration of a session can vary depending on the context. Here are a few common examples:
1. **Online Meetings or Webinars**: Typically last from 30 minutes to several hours, depending on the agenda.
2. **Therapy or Counseling Sessions**: Usually last 45 to 60 minutes.
3. **Classroom Sessions**: Can range from 45 minutes to 2 hours, depending on the institution and subject.
4. **Workshops or Training Sessions**: Often last 1 to 3 hours.
5. **Video Calls**: Can be as short as 10 minutes or as long as several hours.
If you're referring to a specific type of session (e.g., a therapy session, a Zoom meeting, a class), please provide more details so I can give you a more accurate answer.
(B) Basic RAG : The session lasts **30 minutes**.

## Explanation:
```

Q5 and Q6 –refused, not answered

```
The HW3 idle timeout is 5 minutes [3]. [kept 3 of 3 chunks]
Q5 (not in documents)
QUESTION: What was the p99 latency of the fixed endpoint at page size 200 in the second measurement run?
[1] score=0.6640 source=04_n_plus_one.txt chunk_id=04_n_plus_one.txt#2
easured requests. The number of SQL statements per request was read from the X-SQL-Count response header, which does not count the one session check query. Latency was measured with a timer around each...
[2] score=0.6227 source=04_n_plus_one.txt chunk_id=04_n_plus_one.txt#3
or naive and 1.67 ms for fixed at size 10, 7.38 ms and 1.56 ms at size 50, and 25.13 ms and 2.43 ms at size 200. The speed-up was 2.3 times, 4.7 times and 10.3 times. The speed-up grows because every ...
[3] score=0.5792 source=03_database_setup.txt chunk_id=03_database_setup.txt#2
Before the index, EXPLAIN showed a table scan with an estimated cost of 514 over about 5877 rows. After the index, EXPLAIN showed an index lookup with an estimated cost of 0.35 and 1 row....
(A) No RAG : To answer your question about the **p99 latency** of the fixed endpoint at **page size 200** in the **second measurement run**, I would need access to the specific data or results from that measurement run.
Since I don't have access to external data or previous measurement runs, I can't provide the exact value. If you can share the relevant data or results (e.g., a table, graph, or log), I'd be happy to help you extract the p99 latency for that specific case.
(B) Basic RAG : The question asks for the **p99 latency of the fixed endpoint at page size 200 in the second measurement run**.
From the context:
- For **page size 200**, the **latency** for the **fixed endpoint** is given as **2.43 ms**.
- The **p99 latency** is **computed from the 30 values**, and in this case, it is **reported as 2.43 ms**.
Therefore, the **p99 latency of the fixed endpoint at page size 200** is:
**Answer: 2.43 ms**.
(C) Context RAG : I cannot answer this question from the provided documents. [kept 3 of 3 chunks]
=====
Q6 (unrelated)
QUESTION: What is the capital of France?
[1] score=0.3994 source=03_database_setup.txt chunk_id=03_database_setup.txt#1
le holds the session token as its id, the user_id, created_at and expires_at. The trial_sites table holds test data, with an id, a trial_id that points to a trial, a site_name and a city. The databases...
[2] score=0.3729 source=05_react_client.txt chunk_id=05_react_client.txt#0
React client for the Clinical Trial Registry The frontend is a React app built with Vite and React Router. It has five pages: Login.jsx, Home.jsx, CreateRecord.jsx, UpdateRecord.jsx and DeleteRecord....
[3] score=0.3692 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_session_base26, and S...
(A) No RAG : The capital of France is Paris.
(B) Basic RAG : Paris.
(C) Context RAG : I cannot answer this question from the provided documents. [kept 0 of 3 chunks]
saved reports/hw04/raw/three_config_results.json
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % cp reports/hw04/raw/three_config_results.json reports/hw04/raw/three_config_results_v4_final.json
cp reports/hw04/raw/retrieval_and_comparison_log.txt reports/hw04/raw/retrieval_and_comparison_log_v4_final.txt
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

Sweep the context size: run at least one question at $k = 1, 3$, and 5

It asks Q2 and Q3 with 1, 3 and 5 retrieved chunks, and shows how the answer changes as more context is added.

```
(venv) rutujasast@rutujas-MacBook-Pro data260-5825 % python code/rag_sweep.py | tee reports/hw04/raw/hk_sweep_log.txt

Q2 k=1
QUESTION: What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?
[1] score=0.8052 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...
(B) Basic RAG : **Name of the HW4 session cookie:** The question does not provide specific information about the HW4 session cookie, so we cannot determine its name based on the given context.
**HW3 idle timeout:** The HW3 idle timeout is **5 minutes (300 seconds)**..
(C) Context RAG : The name of the HW4 session cookie is not provided in the context [1]. The HW3 idle timeout is 5 minutes (max_age of 300 seconds) [1]. [kept 1 of 1 chunks]

Q2 k=3
QUESTION: What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?
[1] score=0.8052 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...
[2] score=0.7954 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[3] score=0.6539 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
(B) Basic RAG : The name of the HW4 session cookie is **session_id**..
The HW3 idle timeout is **5 minutes (300 seconds)**..
(C) Context RAG : The name of the HW4 session cookie is session_id [3]. The HW3 idle timeout is 5 minutes [1]. [kept 3 of 3 chunks]

Q2 k=5
QUESTION: What is the name of the HW4 session cookie, and how long is the HW3 idle timeout?
[1] score=0.8052 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...
[2] score=0.7954 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[3] score=0.6539 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
[4] score=0.6200 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#1
e is marked HttpOnly, SameSite=lax and Secure. The dashboard page sends the header Cache-Control: no-store, and a small pageshow script reloads the page when the browser back-forward cache restores it...
[5] score=0.6264 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_session_based26, and S...
(B) Basic RAG : The name of the HW4 session cookie is **session_id**.. and the HW3 idle timeout is **300 seconds (5 minutes)**..
(C) Context RAG : The name of the HW4 session cookie is session_id [3]. The HW3 idle timeout is 5 minutes [1]. [kept 5 of 5 chunks]

Q3 k=1
QUESTION: How do the HW3 and HW4 sessions differ in where the session data is stored?
[1] score=0.7324 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_session_based26, and S...
(B) Basic RAG : **Answer:**
In HW3, session data was likely stored in a cookie on the client side (browser), whereas in HW4, session data is stored in a database table (the 'sessions' table), with the session token acting as the primary key and the 'user_id' linking the session to a user. This change reflects a shift from client-side session storage to server-side session storage using a database.
(C) Context RAG : The context provided does not contain information about HW3 sessions. The context only describes the database setup for HW4, mentioning that session data is stored in the sessions table [1]. There is no information provided about where session data is stored for HW3.
I cannot answer this question from the provided documents. [kept 1 of 1 chunks]

Q3 k=3
QUESTION: How do the HW3 and HW4 sessions differ in where the session data is stored?
[1] score=0.7324 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_session_based26, and S...
[2] score=0.7196 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...
[3] score=0.6675 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
(B) Basic RAG : **Answer:**
In **HW3**, session data is stored in a **signed cookie** named 's5825_session' on the client side (browser), while in **HW4**, session data is stored in a **database table** (the 'sessions' table) on the server side.
(C) Context RAG : HW3 stores session data in a signed cookie named s5825_session [2]. HW4 stores session data in a database table named sessions [1]. [kept 3 of 3 chunks]

Q3 k=5
QUESTION: How do the HW3 and HW4 sessions differ in where the session data is stored?
[1] score=0.7324 source=03_database_setup.txt chunk_id=03_database_setup.txt#0
Database setup for HW4 The application stores its data in a MySQL database named s5825_rel, where s5825 is the student prefix. The connection object in database.py is named db_session_based26, and S...
[2] score=0.7196 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#0
Session handling in the HW3 login pages The HW3 pages (/login, /dashboard and /logout) use Starlette SessionMiddleware. The session data is stored inside a signed cookie named s5825_session instead o...
[3] score=0.6675 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#0
Session authentication in the Clinical Trial Registry (HW4) When a user logs in with an email and a password, the FastAPI backend checks the password against a bcrypt hash stored in the users table. ...
[4] score=0.6635 source=01_session_auth_hw4.txt chunk_id=01_session_auth_hw4.txt#2
d, the server answers 401 with the message Session expired or invalid, and an expired row is deleted. A HW4 session lasts 30 minutes on the server. Logging out deletes the session row and clears the ...
[5] score=0.6226 source=02_session_hw3_legacy.txt chunk_id=02_session_hw3_legacy.txt#1
e is marked HttpOnly, SameSite=lax and Secure. The dashboard page sends the header Cache-Control: no-store, and a small pageshow script reloads the page when the browser back-forward cache restores it...
(B) Basic RAG : **Answer:**
In **HW3**, session data is stored in a **signed cookie** named 's5825_session', which is sent to the browser. This cookie is **not encrypted**, so its contents can be read by the user, but tampering is detected due to the signature.
In **HW4**, session data is stored in a **database table** called 'sessions', which contains the session token, user ID, creation time, and expiry time. The session token is stored **ser
```

- k=1 provides too little information. Q2 and Q3 require facts from two different chunks or documents. With only one chunk, Basic RAG guessed that HW3 was “likely stored in a cookie” which was incorrect.
- k=3 provides enough information to answer both questions completely and correctly.

- k=5 does not improve accuracy. It only adds less relevant information, makes the prompt longer, and sometimes produces shorter answers.
- Evaluation table and summary (accuracy, faithfulness, format compliance, robustness)

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % python code/rag_eval.py
```

| Q | Config | Correct retrieval | Correct answer | Grounded | Refused when needed | Format OK |
|----|--------|-------------------|----------------|----------|---------------------|-----------|
| Q1 | A | n/a | 0 | 0 | n/a | 0 |
| Q1 | B | 1 | 1 | 1 | n/a | 0 |
| Q1 | C | 1 | 1 | 1 | n/a | 1 |
| Q2 | A | n/a | 0 | 0 | n/a | 0 |
| Q2 | B | 1 | 1 | 1 | n/a | 0 |
| Q2 | C | 1 | 1 | 1 | n/a | 1 |
| Q3 | A | n/a | 0 | 0 | n/a | 0 |
| Q3 | B | 1 | 1 | 1 | n/a | 0 |
| Q3 | C | 1 | 1 | 1 | n/a | 1 |
| Q4 | A | n/a | 0 | 0 | n/a | 0 |
| Q4 | B | 1 | 1 | 1 | n/a | 0 |
| Q4 | C | 1 | 1 | 1 | n/a | 1 |
| Q5 | A | n/a | 1 | 1 | 1 | 0 |
| Q5 | B | n/a | 0 | 0 | 0 | 0 |
| Q5 | C | n/a | 1 | 1 | 1 | 1 |
| Q6 | A | n/a | 0 | 0 | 0 | 0 |
| Q6 | B | n/a | 0 | 0 | 0 | 0 |
| Q6 | C | n/a | 1 | 1 | 1 | 1 |

| Config | Accuracy (correct answer) | Faithfulness (grounded) | Format compliance | Robustness (Q5+Q6 refused) |
|--------|---------------------------|-------------------------|-------------------|----------------------------|
| A | 1/6 | 1/6 | 0/6 | 1/2 |
| B | 4/6 | 4/6 | 0/6 | 0/2 |
| C | 6/6 | 6/6 | 6/6 | 2/2 |

```
(venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %
```

- Config C performed best, scoring 6/6 for accuracy, reliable answers, and proper formatting. It also correctly refused to answer Q5 and Q6 when the documents did not provide enough information.
- Config B scored 4/6. It answered Q1–Q4 but made mistakes on Q5 and Q6.
- Config A scored only 1/6 because it had no documents. It often gave vague or invented answers. Its only correct response was refusing Q5 because it needed more data.

Short analysis :

Retrieval: The system found the correct information for Q1–Q4. Q2 and Q3 required facts from two different documents, and both supporting facts appeared in the top three retrieved chunks. Q4 was an ambiguous question, but the system successfully retrieved information about both HW4 and HW3 sessions. For Q5, the system found related latency information, but none of the retrieved chunks contained the requested p99 value. For Q6, the question was unrelated to the documents, so the retrieval scores were low.

What helped: The value of k had the biggest effect on the results. When k=1, the system retrieved only one chunk, so Q2 and Q3 were incomplete because each question required information from two sources. When k=3, the system retrieved enough information to answer both questions correctly. Increasing the value to k=5 did not improve the answers. Instead, it added less relevant chunks, made the

prompt longer, and gave the model more information to process. Therefore, k=3 provided the best balance between completeness and relevance.

The chunk size also helped. Using chunks of 500 characters with a 50-character overlap kept related facts together and reduced the chance of splitting important information between chunks. Removing chunks with scores below 0.55 also helped the system reject Q6 because no sufficiently relevant evidence remained.

Hallucinations: Basic RAG incorrectly answered Q6 with “Paris.” It also treated the median latency value as the p99 value in Q5, even though the documents never provided a p99 measurement. The no-RAG setup performed worse because it had no documents to guide the model. It gave an invented Express.js answer for Q3 and also answered the unrelated Q6 question. Context RAG correctly refused to answer both Q5 and Q6 because the documents did not contain enough evidence.

What did not help: Prompt-based grounding rules alone were not enough to prevent the Q5 mistake. A stricter second version of the rules and an additional language-model check also failed. The local Qwen3:8B model saw a related latency number and assumed it answered the question. The problem was fixed only after adding a simple Python validation check. This check looks for the specific metric mentioned in the question. If the question asks for p99 but the retrieved text does not contain p99 information, the system refuses to answer.

How the three parts work together: Retrieval makes sure the necessary evidence is available. Context engineering removes distractions and organizes the evidence clearly. Prompt rules guide the model to stay within the documents. However, code-based checks provide an additional safety layer when the model misunderstands related information. The strongest results came from combining good retrieval, clean context, clear instructions, and automatic validation.

Limitations: Only six questions were tested, and the answers were evaluated manually. Therefore, the results show a clear pattern but cannot be treated as a statistically conclusive evaluation. Q4 also revealed a remaining weakness: although the system provided both durations, it did not clearly explain which duration belonged to HW3 and which belonged to HW4.

Verification: Tested all the 10 tests

```

● (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 % python code/verify_hw4.py
PASS - backend responds on port 8425
PASS - login succeeds ('-', 'status=200')
PASS - naive endpoint returns 5 trials ('-', 'status=200')
PASS - fixed endpoint returns 5 trials ('-', 'status=200')
PASS - fixed endpoint uses fewer SQL statements than naive ('-', 'naive=51 fixed=1')
PASS - unauthenticated request rejected ('-', 'status=401')
PASS - corpus has 5 documents
PASS - three_config_results.json exists
PASS - k_sweep_results.json exists
PASS - evaluation_table.csv exists

All passed: True
saved reports/hw04/verification.json
○ (venv) rutujasaste@Rutujas-MacBook-Pro data260-5825 %

```

AI_use:

```
# AI_USE.md
```

1. What I used an AI assistant for, and what I did myself

I used Claude code to understand RAG and also write the RAG scripts for Part 4. Also for understanding the React coding I used W3 schools and chatgpt to get easier explanation of how it works and for the scripts to understand. For frontend took help of AI to correct the scripts if wrong. Also to understand the commands and execute them for RAG took help of AI.

I did the following myself: installed and configured MySQL, wrote the scripts for frontend by understanding using different websites and AI as well, created the database and ran the seed script, ran every Postman request, read every generated answer in configs A/B/C to check it against the source documents, and wrote the final analysis text based on my own reading of the results.

2. One AI-produced output that was wrong or unsuitable

In Part 4, my first version of the context-engineered RAG prompt (grounding rules v1) answered Q5 ("What was the p99 latency of the fixed endpoint at page size 200 in the second measurement run?") with "2.43 ms". This was wrong: the documents only state a median latency of 2.43ms at page size 200, and never state a p99 value or mention a "second run". The model treated a related number as if it answered the question.

3. How I detected the problem

I compared the generated answer against the actual source document (04_n_plus_one.txt) and confirmed it does not contain the word "p99" anywhere near the 2.43 ms figure, and never mentions a second measurement run at all. The assignment requires Q5 to be refused, so I knew this was a failure even before checking the wording closely.

4. What I changed and why it works now

I first tried making the grounding instructions stricter in two versions, v1 and v2. I also tested an LLM-based verification step in v3, where the model reviewed its own context before answering. However, none of these methods prevented the hallucination. The small local model continued to treat the related median value as close enough to the requested answer, even though the question specifically asked for p99.

The solution was to add a simple Python check before sending the question to the LLM. This check looks for the exact metric mentioned in the question, such as p99, p95, p50, median, mean, or the word "second." It then searches the retrieved text to see whether that exact term is present.

If the requested term is missing from the retrieved chunks, the system refuses to answer instead of guessing. This approach is deterministic because it is handled by code rather than depending on the model to judge whether the evidence is sufficient.

After adding this check, the system correctly refused Q5 because the retrieved documents did not contain a p99 value. The other questions, Q1–Q4 and Q6, were not affected. This shows that the code-level validation successfully addressed the hallucination while allowing questions with adequate supporting evidence to continue normally.